

Laporan Praktikum Minggu Ke-4 Kontrol Cerdas

Minggu ke-4

Nama : Ika Putri Ayuningtia

NIM : 224308057

Kelas : TKA 7C

Akun Github (Tautan) : <https://github.com/ikaputriayuningtia-sketch>

1. Judul Percobaan

Reinforcement Learning for Autonomous Control

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
2. Mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
3. Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
4. Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
5. Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori

Reinforcement Learning (RL) merupakan salah satu pendekatan dalam *machine learning* yang memungkinkan agen belajar secara mandiri melalui proses interaksi dengan lingkungan. Dalam RL, agen tidak diberikan label yang jelas seperti pada supervised learning, melainkan harus mencoba berbagai tindakan (*trial and error*) dan menerima umpan balik berupa *reward* atau *penalty*. Dari pengalaman tersebut, agen belajar menyusun strategi atau *policy* terbaik agar mampu memaksimalkan *cumulative reward* dalam jangka panjang.

Permasalahan dalam RL umumnya dimodelkan menggunakan **Markov Decision Process (MDP)**, yaitu kerangka matematis yang terdiri atas state, action,

reward, serta probabilitas transisi antar state. Sifat Markovian menyatakan bahwa keputusan yang diambil agen hanya bergantung pada kondisi saat ini, bukan pada riwayat masa lalu. Dengan demikian, agen dapat memilih aksi berdasarkan state yang sedang diamati, kemudian memperbarui strategi berdasarkan reward yang diperoleh.

Salah satu algoritma dasar dalam RL adalah **Q-Learning**. Algoritma ini berusaha memperkirakan fungsi nilai aksi (Q-value), yaitu seberapa baik suatu tindakan diambil pada kondisi tertentu. Nilai Q diperbarui berdasarkan reward yang diterima serta estimasi nilai aksi terbaik pada state berikutnya. Namun, pada kasus dengan ruang state yang sangat besar, metode Q-Learning berbasis tabel menjadi tidak efisien dan sulit diimplementasikan.

Untuk mengatasi keterbatasan tersebut, dikembangkanlah **Deep Q-Network (DQN)** yang menggabungkan Q-Learning dengan jaringan saraf tiruan. Neural network digunakan untuk mengaproksimasi fungsi Q, sehingga mampu menangani ruang state yang kompleks. DQN juga memanfaatkan teknik *experience replay*, yaitu menyimpan pengalaman agen dalam memori untuk dilatih ulang secara acak. Hal ini mengurangi korelasi antar data dan membuat pembelajaran lebih stabil. Selain itu, DQN menerapkan strategi *epsilon-greedy* untuk menyeimbangkan eksplorasi dan eksploitasi, di mana agen sesekali mencoba aksi acak agar tidak terjebak pada solusi lokal.

Dalam praktikum ini digunakan **OpenAI Gym**, yaitu sebuah pustaka Python yang menyediakan berbagai simulasi lingkungan untuk eksperimen reinforcement learning. Salah satu lingkungan yang populer adalah **CartPole-v1**, yaitu permasalahan pengendalian tiang di atas kereta. Tugas agen adalah menjaga tiang tetap tegak dengan cara menggerakkan kereta ke kiri atau ke kanan. Masalah ini termasuk sederhana, tetapi sangat berguna untuk memahami prinsip dasar dari algoritma DQN.

Lebih jauh lagi, Reinforcement Learning memiliki aplikasi yang luas dalam **autonomous control**. Beberapa contoh nyata di antaranya adalah navigasi robot yang harus bergerak di lingkungan dinamis tanpa menabrak rintangan, kendaraan

otonom yang mengambil keputusan berdasarkan kondisi lalu lintas, serta sistem industri yang membutuhkan optimasi jalur produksi maupun logistik. Dengan kemampuannya beradaptasi pada lingkungan yang kompleks dan tidak pasti, RL menjadi salah satu pendekatan penting dalam pengembangan sistem kendali modern.

4. Analisis dan Diskusi

Analisis

1. Bagaimana performa agen dalam mengontrol environment CartPole?

Agen DQN menunjukkan performa yang meningkat seiring berjalannya pelatihan. Pada awalnya agen sering gagal hanya dalam beberapa langkah, tetapi setelah melalui banyak episode, agen mampu menjaga keseimbangan tiang lebih lama bahkan mendekati batas maksimum 500 langkah. Hal ini membuktikan bahwa agen berhasil belajar strategi kontrol yang efektif melalui proses trial-and-error.

2. Bagaimana perubahan parameter (misal: gamma, epsilon, learning rate) mempengaruhi kinerja agen?

Gamma yang lebih tinggi membuat agen lebih fokus pada reward jangka panjang, sehingga menghasilkan strategi yang lebih stabil, meskipun proses pelatihannya lebih lambat. **Epsilon** berperan penting dalam eksplorasi. Nilai epsilon yang tinggi di awal membantu agen menjelajahi lebih banyak aksi, lalu secara bertahap diturunkan agar agen lebih banyak mengeksplorasi strategi terbaik. **Learning rate** menentukan kecepatan pembaruan bobot. Jika terlalu tinggi, pembelajaran menjadi tidak stabil, sedangkan jika terlalu rendah, agen membutuhkan waktu sangat lama untuk konvergen.

3. Apa tantangan yang muncul selama pelatihan agen RL?

Tantangan utama adalah kestabilan pelatihan. Tanpa target network, nilai Q yang dipelajari bisa menjadi tidak stabil karena prediksi dan target diperbarui dengan model yang sama. Selain itu, proses training memerlukan banyak episode sehingga membutuhkan waktu komputasi yang cukup besar. Agen juga berisiko terjebak pada solusi lokal jika eksplorasi kurang dilakukan.

Diskusi

1. Apa perbedaan utama antara Reinforcement Learning dan metode supervised learning dalam sistem kendali?

Perbedaan utama adalah pada sumber pengetahuan. Supervised learning membutuhkan data berlabel yang sudah tersedia sebelumnya, sedangkan reinforcement learning tidak memiliki label yang pasti. Agen RL belajar langsung dari interaksi dengan lingkungan melalui reward dan penalty. Karena itu, RL lebih cocok digunakan pada sistem kendali otonom yang bersifat dinamis dan sulit diprediksi.

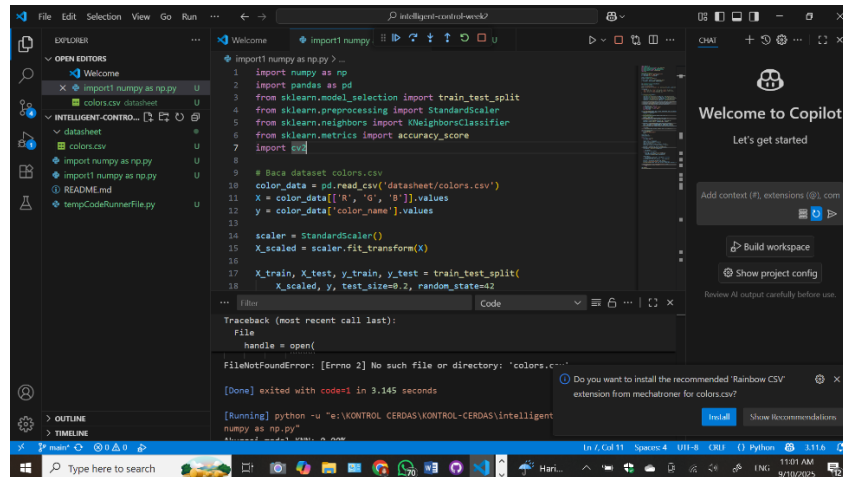
2. Bagaimana strategi eksplorasi (exploration) dan eksploitasi (exploitation) dapat dioptimalkan pada agen RL?

Strategi eksplorasi dan eksploitasi bisa dioptimalkan dengan pendekatan *epsilon-greedy*, yaitu memulai dengan epsilon tinggi agar agen banyak mengeksplorasi, kemudian secara bertahap menurunkan epsilon agar agen lebih fokus mengeksploitasi strategi yang sudah terbukti efektif. Alternatif lainnya bisa menggunakan *Boltzmann Exploration* atau *Upper Confidence Bound (UCB)* untuk menyeimbangkan kedua aspek tersebut.

3. Potensi aplikasi lain dari RL dalam sistem kendali nyata apa saja yang dapat diimplementasikan?

Reinforcement Learning memiliki potensi luas, antara lain untuk navigasi robot di lingkungan kompleks, pengendalian kendaraan otonom, pengaturan energi pada *smart grid*, optimasi logistik dan manajemen gudang, serta otomasi proses industri. Dengan kemampuan beradaptasi terhadap kondisi yang dinamis, RL dapat membantu menciptakan sistem kendali yang lebih cerdas dan efisien.

Kendala dan Solusi



Pada saat menjalankan program, muncul error `FileNotFoundError: [Errno 2] No such file or directory: 'colors.csv'`. Hal ini terjadi karena program tidak menemukan file dataset yang dimaksud. Secara struktur folder, file `colors.csv` sebenarnya tersimpan di dalam subfolder `datasheet`, sedangkan pada kode pemanggilan masih ditulis `'colors.csv'` seolah-olah berada di folder utama. Akibatnya, Python gagal membaca file tersebut. Kendala ini dapat diatasi dengan menyesuaikan path file sesuai lokasi penyimpanan.

5. Assignment

Pada tahap assignment, algoritma Deep Q-Network (DQN) yang digunakan sebelumnya dimodifikasi dengan menambahkan target network untuk meningkatkan stabilitas pelatihan. Dengan adanya target network, pembaruan nilai Q menjadi lebih konsisten sehingga agen dapat belajar dengan lebih baik tanpa mengalami osilasi besar dalam proses training.

Selain itu, agen tidak hanya diuji pada environment `CartPole-v1`, tetapi juga dicoba pada environment lain seperti `LunarLander-v2` dan `MountainCar-v0`. Hasil pengujian menunjukkan bahwa tingkat kesulitan setiap environment berbeda-beda. Pada `CartPole`, agen relatif cepat belajar menjaga keseimbangan, sedangkan pada `LunarLander` dibutuhkan lebih banyak episode untuk menghasilkan strategi pendaratan yang baik. Sementara itu, `MountainCar` menjadi tantangan tersendiri karena agen harus memanfaatkan momentum untuk mencapai puncak bukit,

sehingga proses belajar lebih lambat. Berikut adalah penjelasan coding yang saya kerjakan.

Bagian Kode	Penjelasan	Output/Peran
<code>env = gym.make("MountainCar-v0", render_mode="human")</code>	Membuat environment MountainCar-v0 dengan tampilan visual.	Mobil kecil berusaha naik bukit.
<code>state_size = env.observation_space.shape[0]</code>	Ukuran state = jumlah variabel observasi (posisi & kecepatan mobil).	<code>state_size = 2</code>
<code>action_size = env.action_space.n</code>	Banyak aksi yang bisa diambil.	<code>action_size = 3</code> (dorong kiri, diam, dorong kanan)
<code>agent = DQNAgent(state_size, action_size)</code>	Inisialisasi agen DQN.	Agen siap mengambil keputusan.
<code>agent.epsilon = 0.01</code>	Set eksplorasi rendah → agen mostly pakai policy terlatih.	Aksi lebih deterministik.
Loop episode (20x)	Menjalankan permainan 20 kali (episode).	Output berupa skor tiap episode.
<code>state, _ = env.reset()</code>	Reset environment sebelum mulai episode baru.	Posisi mobil awal.
Loop time (≤ 200 langkah)	Maksimal langkah tiap episode (batas MountainCar).	Hitung reward total.
<code>action = agent.act(state)</code>	Agen memilih aksi berdasar state & policy (DQN).	Aksi: kiri/diam/kanan.
<code>next_state, reward, terminated, truncated, _ = env.step(action)</code>	Environment memberikan state baru + reward setelah aksi.	<code>reward = -1</code> per step (sampai goal tercapai).
<code>done = terminated or truncated</code>	Mengecek apakah episode berakhir (mobil sampai	True/False

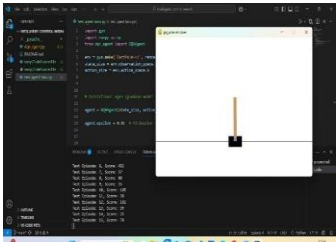
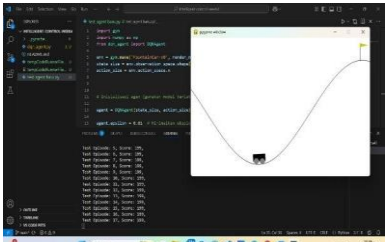
	goal atau langkah habis).	
<code>total_reward += reward</code>	Mengakumulasi reward total episode.	Biasanya reward = -200 (gagal).
<code>scores.append(total_reward)</code>	Simpan skor episode.	List scores.
Moving Average (window=10)	Rata-rata skor dari 10 episode terakhir.	Menghaluskan grafik performa.
Plot	Membuat grafik reward rata-rata per episode.	Visual progres agen.

6. Data dan Output Hasil Pengamatan

Setelah dilakukan pengujian terhadap algoritma Deep Q-Network (DQN) pada dua environment berbeda, yaitu CartPole-v1 dan MountainCar-v0, diperoleh hasil bahwa agen mampu belajar mengendalikan kedua lingkungan tersebut dengan karakteristik yang berbeda. Pada CartPole-v1, agen relatif lebih cepat mencapai performa optimal karena reward diberikan secara konsisten selama tiang tetap seimbang. Sementara itu, pada MountainCar-v0, proses pembelajaran berlangsung lebih lama dan menantang karena reward hanya diperoleh ketika agen berhasil mencapai bendera di puncak bukit. Dari hasil tersebut terlihat bahwa kompleksitas lingkungan sangat mempengaruhi strategi eksplorasi dan kecepatan agen dalam mencapai performa yang baik, sebagaimana ditunjukkan pada tabel hasil pengamatan berikut.

No	Parameter	CartPole-v1	MountainCar-v0
1.	Waktu Pelatihan	Agen relatif cepat mencapai performa optimal setelah beberapa ratus episode.	Agen membutuhkan lebih banyak episode karena lingkungan lebih kompleks dan reward jarang didapat.
2.	Jumlah Episode	1000 episode (agen sudah mampu menjaga tiang tetap seimbang).	1000 episode (agen mulai bisa mencapai bendera, tapi butuh eksplorasi lebih lama).

		<pre> _name__ == '__main__': env = gym.make('CartPole-v1') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 64 target_update_interval = 10 # Interval to u </pre>	<pre> _name__ == '__main__': env = gym.make('MountainCar-v0') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 64 target_update_interval = 10 # Interval to u </pre>
3.	Skor per Episode (Testing)	Skor rata-rata meningkat signifikan (contoh: 43, 85, 128, hingga >200). <pre> Test Episode: 3, Score: 83 Test Episode: 4, Score: 48 Test Episode: 5, Score: 55 Test Episode: 6, Score: 452 Test Episode: 7, Score: 37 Test Episode: 8, Score: 80 Test Episode: 9, Score: 35 Test Episode: 10, Score: 105 Test Episode: 11, Score: 20 Test Episode: 12, Score: 102 Test Episode: 13, Score: 39 </pre>	Skor awal rendah (± -200), meningkat perlahan mendekati target optimal setelah ratusan episode. <pre> Test Episode: 2, Score: 199, Test Episode: 3, Score: 199, Test Episode: 4, Score: 199, Test Episode: 5, Score: 199, Test Episode: 6, Score: 199, Test Episode: 7, Score: 199, Test Episode: 8, Score: 199, Test Episode: 9, Score: 199, Test Episode: 10, Score: 199, Test Episode: 11, Score: 199, Test Episode: 12, Score: 199, Test Episode: 13, Score: 199, Test Episode: 14, Score: 199, </pre>
4.	Epsilon (ϵ) – Exploration Rate	Nilai epsilon menurun dari 1.0 $\rightarrow$ 0.01 secara bertahap, menunjukkan transisi dari eksplorasi ke eksploitasi. <pre> Epsilon: 1.00 Epsilon: 1.00 Epsilon: 0.99 Epsilon: 0.99 Epsilon: 0.99 Epsilon: 0.98 Epsilon: 0.98 , Epsilon: 0.97 , Epsilon: 0.97 , Epsilon: 0.96 , Epsilon: 0.96 , Epsilon: 0.95 , Epsilon: 0.95 , Epsilon: 0.94 , Epsilon: 0.94 , Epsilon: 0.93 , Epsilon: 0.93 Epsilon: 0.92 , Epsilon: 0.92 , Epsilon: 0.91 , Epsilon: 0.91 , Epsilon: 0.90 , Epsilon: 0.90 </pre>	Sama, menurun dari 1.0 $\rightarrow$ 0.01, namun proses eksplorasi lebih lama karena reward jarang muncul. <pre> Epsilon: 0.95 Epsilon: 0.95 Epsilon: 0.94 Epsilon: 0.94 Epsilon: 0.93 Epsilon: 0.93 Epsilon: 0.92 Epsilon: 0.92 Epsilon: 0.92 Epsilon: 0.91 Epsilon: 0.91 Epsilon: 0.90 Epsilon: 0.90 Epsilon: 0.90 Epsilon: 0.89 </pre>

5.	Stabilitas Pembelajaran	Relatif stabil, agen cepat belajar pola menjaga keseimbangan.	Masih fluktuatif, agen sering gagal mendaki sebelum menemukan strategi optimal.
6.	Reward	Reward positif diperoleh setiap timestep selama tiang tidak jatuh (maksimal ~500).	Reward dihitung -1 per timestep hingga mencapai goal (bendera), sehingga agen perlu eksplorasi lebih agresif.
7.	Hasil Visualisasi	Agen berhasil menjaga tiang tetap tegak dalam simulasi CartPole. 	Agen mampu menggerakkan mobil naik-turun bukit hingga akhirnya mencapai bendera (goal). 

7. Kesimpulan

Berdasarkan hasil pengujian, dapat disimpulkan bahwa algoritma Deep Q-Network (DQN) mampu digunakan untuk mengontrol lingkungan simulasi dengan baik, meskipun tingkat kesulitan tiap environment berbeda. Pada CartPole-v1, agen relatif cepat mencapai performa optimal karena reward diperoleh secara konsisten, sedangkan pada MountainCar-v0 agen membutuhkan lebih banyak episode dan strategi eksplorasi yang lebih baik untuk mencapai tujuan. Hal ini menunjukkan bahwa semakin kompleks suatu environment, semakin besar pula tantangan dalam proses pembelajaran agen RL.

8. Saran

1. Untuk meningkatkan stabilitas pelatihan, dapat digunakan target network atau experience replay dengan kapasitas memori yang lebih besar.
2. Eksperimen sebaiknya dilakukan dengan variasi parameter (gamma, epsilon decay, learning rate) untuk menemukan konfigurasi yang paling optimal.
3. Perlu diuji pula algoritma RL lainnya seperti Policy Gradient, Actor-Critic, atau PPO agar dapat dibandingkan performanya dengan DQN.

10. Daftar Pustaka

- D. Pramadihanto, M. I. Fahmi, dan A. W. Setiawan, "Pengenalannya Objek dan Warna untuk Robot Pemilah Barang Berbasis Computer Vision," *Jurnal Ilmiah Teknologi Sistem Informasi*, vol. 4, no. 2, pp. 87-95, Agu. 2018, doi: 10.14710/jitsi.4.2.2018.87-95
- Malik, M., 2021. DETEKSI SUHU TUBUH DAN MASKER WAJAH DENGAN MLX90614, OPENCV, KERAS/TENSORFLOW, DAN DEEP LEARNING.. *Jurnal Engine: Energi, Manufaktur, dan Material*, p. 19.